Music Library API: полный разбор

Лабораторная работа 4 «REST-сервис на FastAPI и клиент к нему». Что это за программа, какие понятия нужны, что происходит при запуске и как устроен каждый файл. Номера строк соответствуют коммиту `21489c9`. В коде зелёным написаны пояснения к строкам; в самих файлах комментариев меньше.

0. Что это такое
1. Понятия, без которых код не понять
2. Карта файлов
3. Что происходит при запуске
4.1. app/schemas.py: описание трека
4.2. app/repository.py: хранилище
4.3. app/main.py: адреса API
4.4. client/text.py: перевод в ASCII
4.5. client/api.py: клиент на httpx
4.6. client/cli.py: командная строка
4.7. client/__main__.py: точка входа
4.8. tests/conftest.py: заготовки для тестов
4.9. tests/test_api.py: тесты адресов
4.10. tests/test_repository.py
4.11. tests/test_client.py
4.12. Настройки: pyproject, requirements, .gitignore
5. Как это связано с заданием
6. Что сказать на защите

0. Что это такое

Music Library API — программа, которая хранит **список треков**: название, исполнители, альбом, длительность, год. Музыку она не играет и не скачивает. Это как плейлист, только без самой музыки — одни записи.

Программа состоит из двух половинок:

- **Сервер** (папка `app/`) хранит список в файле базы данных и ждёт вопросов по сети: «добавь трек», «покажи все», «найди по названию», «измени», «удали». Сам он ничего не печатает и ни о чём не спрашивает.
- **Клиент** (папка `client/`) — программа для командной строки, через которую эти вопросы задаёт человек: `python -m client add ...`, `python -m client list`. Вместо клиента можно открыть в браузере страницу `/docs` — там те же действия кнопками.

Сервер и клиент могут быть на разных компьютерах: сервер на VPS, клиент у тебя на ноутбуке. Список общий — что добавил с одного устройства, видно со всех.

Зачем это лабе. Лаба не про музыку. Она про то, как устроен **любой веб-сервис**. Когда ты ставишь лайк в Instagram, телефон отправляет серверу запрос «сохрани лайк», сервер записывает его в базу и отвечает «готово». Мы делаем такой сервер сами — маленький, на одну сущность (трек), — и клиента к нему, который не зависает и не падает с трейсбеком, если сервер сломался или недоступен. Треки взяты из ЛР3: ЛР3 выгружает их из Spotify в JSON, а клиент ЛР4 умеет этот JSON загрузить на сервер.

1. Понятия, без которых код не понять

Аналогия: **кафе**. Кухня принимает заказы и готовит, посетитель заказывает.

Понятие	В кафе	В программе
Сервер	кухня: принимает заказы, готовит, отдаёт	программа, которая ждёт запросов по сети и отвечает на них (`app/`)
Клиент	посетитель, который делает заказ	программа или браузер, которые отправляют запросы (`client/`)
HTTP	язык, на котором пишут заказ: «что сделать» + «с чем» + подробности	текстовый протокол поверх TCP: метод, адрес, заголовки, тело. Ответ: код, заголовки, тело

Метод	глагол заказа: «покажите», «добавьте», «замените», «уберите»	`GET` — прочитать, `POST` — создать, `PUT` — заменить, `DELETE` — удалить
Адрес (путь)	что именно: «всё меню», «блюдо №5»	`/items` — все треки, `/items/5` — трек номер 5, `/items/search?q=numb` — поиск
Query-параметры	уточнения к заказу: «первые 20 позиций»	после `?` в адресе: `/items?limit=20&offset=0`
Тело (body)	сам бланк заказа	данные в запросе, у нас JSON: `{"title": "Numb", "duration_sec": 187}`
Код ответа	что сказал официант	`200` «вот, держи»; `201` «создано»; `204` «сделано, показывать нечего»; `404` «такого нет»; `422` «бланк заполнен неправильно»; `409` «такое уже есть»; `500` «кухня сломалась»
JSON	форма бланка, которую понимают все	текст вида `{"ключ": значение}`, списки `[...]`, числа, строки, `null`
REST	правила оформления заказов: адрес — это вещь, метод — действие с ней	стиль API: ресурсы с адресами (`/items/5`), действия — методами HTTP, сервер не помнит клиента между запросами
uvicorn	входная дверь и официант: впускает, выслушивает, несёт заказ на кухню	сервер, который слушает порт, читает байты из сокета и превращает их в HTTP-запрос
FastAPI	шеф: смотрит на заказ и отдаёт его нужному повару	фреймворк: по методу и адресу выбирает нашу функцию, достаёт и проверяет данные, упаковывает ответ
Pydantic	контролёр бланков: неправильный бланк сразу возвращает с пометками	библиотека проверки данных по описанию класса; нарушение — ответ `422` со списком полей
SQLite	кладовая — одна толстая тетрадь	база данных в одном файле `data/tracks.db`; переживает перезапуск
Репозиторий	кладовщик: в кладовую ходит только он	класс, где собран весь SQL; остальной код просит его «добавь», «найди», «удали»
httpx	телефон, по которому посетитель звонит в кафе	библиотека Python для отправки HTTP-запросов
Таймаут	«жду ответа 5 секунд, потом кладу трубку»	`timeout=5.0`: без него клиент ждал бы зависший сервер вечно
Ретрай	«было занято — перезвоню»	повтор запроса после сбоя, с растущей паузой
Идемпотентность	«повторите заказ» безопасно для «покажите меню» и опасно для «добавьте пиццу» — будет две	повтор даёт тот же результат: `GET`, `PUT`, `DELETE` — да; `POST` — нет, два POST = два трека

Ещё несколько слов из Python, которые встретятся в коде:

- **Аннотации типов** — подписи «что сюда кладут»: `item_id: int`, `-> ItemRead` («функция вернёт ItemRead»). FastAPI и Pydantic по ним понимают, что проверять, а mypy проверяет, что мы сами их не нарушили.
- `X | None` — «X или ничего». Например, `year: int | None` — год или пусто.
- `Annotated[int, Query(ge=1)]` — «тип int и ещё дополнительные правила»: здесь «взять из query, не меньше 1».
- **Декоратор** `@app.get("/items")` — табличка над функцией: «вызывать меня для GET-запросов на адрес /items».

- **Исключение** (`raise`, `try / except`) — аварийный выход: функция прерывается, ошибка летит вверх по вызовам, пока её кто-нибудь не поймает в `except`.
- `with` — «открыть и гарантированно закрыть»: соединение с базой, клиент httpx. Закроется, даже если внутри была ошибка.
- **Protocol** — описание «что умеет» без кода (как в ЛР3). Подходит любой класс с такими же методами.
- **Dependency** (`Depends`) — просьба к FastAPI: «перед вызовом моей функции подставь в этот аргумент вот это». У нас так подставляется хранилище.
- **Фикстура** (в тестах) — заготовка: тест пишет имя аргумента, pytest сам создаёт для него объект.

2. Карта файлов

```
uvicorn app.main:app            <- запускаешь сервер
        |
        v
app/main.py         адреса API: какая функция отвечает на какой запрос и с каким кодом
   |-- проверяет вход и выход моделями   -> app/schemas.py     (описание трека)
   '-- просит хранилище                   -> app/repository.py  (SQL, файл data/tracks.db)

python -m client ...            <- запускаешь клиент (в другом окне или на другой машине)
        |
        v
client/__main__.py  точка входа: вызывает main() из cli.py
client/cli.py       разбирает команду, печатает результат
   |-- client/api.py       LibraryClient: отправляет HTTP-запросы через httpx
   |      '-- app/schemas.py     те же модели: проверить, что ответил сервер
   '-- client/text.py      to_ascii(): "Группа крови" -> "Gruppa krovi"

tests/                47 тестов pytest: адреса, хранилище, клиент
examples/tracks.json  4 трека в формате экспорта ЛР3
pyproject.toml        настройки муру и pytest
requirements.txt      какие библиотеки ставить
```

Клиент и сервер общаются **только через HTTP**. Общий у них один файл — `app/schemas.py`: обе стороны одинаково понимают, как выглядит трек.

3. Что происходит при запуске

3.1. Запуск сервера: `uvicorn app.main:app --reload`

1. `app.main:app` значит «модуль `app/main.py`, переменная `app` в нём». uvicorn импортирует этот файл.
2. При импорте Python выполняет `main.py` сверху вниз: создаётся объект `app = FastAPI(...)`, а каждый декоратор `@app.get` / `@app.post` записывает свою функцию в таблицу маршрутов «метод + адрес → функция». База данных пока не открывается.
3. uvicorn открывает порт 8000 на `127.0.0.1` и ждёт подключений. С `--host 0.0.0.0` — принимает их и с других машин.
4. `--reload`: при изменении .py-файлов uvicorn перезапускается сам. Это для разработки, на сервере не нужно.

3.2. Команда клиента: `python -m client add --title "Numb" --artist "Linkin Park" --duration 3:07`

На стороне клиента:

1. `-m client` — Python запускает пакет `client`, то есть файл `client/__main__.py`. Тот вызывает `main()` из `cli.py`.

2. `main()` переключает вывод в ASCII и отдаёт аргументы **argparse**. Получается: команда `add`, `title="Numb"`, `artist=["Linkin Park"]`, `duration=187` (функция `_duration` превратила «3:07» в секунды).
3. `run()` видит команду `add` и вызывает `_track_from_args()`: та собирает словарь и проверяет его моделью `ItemCreate`. Если, например, название пустое, — ошибка здесь же, в сеть ничего не уходит.
4. `client.create(track)` → `_request("POST", "/items", json=...)` → httpx отправляет серверу текст:

```
POST /items HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"title": "Numb", "artists": ["Linkin Park"], "album": "", "duration_sec": 187, "year": null, "spotify_id":
null}
```

На стороне сервера:

5. uvicorn принимает соединение, читает байты, разбирает HTTP и передаёт запрос FastAPI.
6. FastAPI ищет в таблице маршрутов «POST + /items» и находит функцию `create_item`.
7. FastAPI смотрит на аргументы `create_item`: `data: ItemCreate` — «возьми тело запроса и проверь моделью»; `repo: Repo` — «вызови `get_repository()`» (в первый раз она откроет базу); `response: Response` — объект, чтобы добавить заголовок.
8. Проверка прошла → FastAPI вызывает `create_item(data, repo, response)`. Внутри `repo.add(data)` делает `INSERT` в SQLite и возвращает трек уже с `id=1`.
9. Функция ставит заголовок `Location: /items/1` и возвращает трек. FastAPI превращает его в JSON и отвечает с кодом 201:

```
HTTP/1.1 201 Created
location: /items/1
content-type: application/json

{"title": "Numb", "artists": ["Linkin Park"], ..., "id": 1, "created_at": "2026-10-18T14:37:41.220453Z", ...}
```

Снова на стороне клиента:

10. httpx получает ответ. `raise_for_status()`: код 201 — не ошибка, идём дальше.
11. `_parse()` проверяет JSON моделью `ItemRead`: сервер ответил тем, что обещал?
12. `cli.py` печатает `created #1 Linkin Park - Numb 3:07` и завершается с кодом 0.

3.3. А если что-то пошло не так

Ситуация	Где ловится	Что увидишь
в команде пустое название	клиент: `ItemCreate` в `_track_from_args`, ещё до сети	`error: invalid input: title: String should have at least 1 character`
кривые данные пришли на сервер (curl, /docs)	сервер: FastAPI + Pydantic, до нашей функции	ответ 422 со списком полей; клиент: `error: invalid input (422): ...`
трека с таким id нет	`get_item`: `repo.get` вернул `None` → `HTTPException(404)`	`error: not found (404): track 5 not found`
сервер выключен	httpx: `ConnectError` → три попытки → `ServiceUnavailableError`	`error: cannot connect to ...; is the server running?`
сервер завис	httpx: прошло 5 с → `ReadTimeout`	`error: ... did not answer within 5 s (ReadTimeout)`

сервер сломался (5xx)	`raise_for_status` → повтор для GET/PUT/DELETE, POST не повторяется	`error: server error (503): ...`

Во всех случаях клиент печатает одну строку в stderr и выходит с кодом 1 — без трейсбека.

4. Файлы по строкам

4.1. Описание трека `app/schemas.py`

Этот файл ничего не *делает*. Он **описывает**, как выглядит трек: какие у него поля и какие для них правила. По этому описанию Pydantic проверяет данные, а FastAPI рисует страницу `/docs`. Такое описание называют **контрактом**: о нём договорились сервер и клиент.

```
from datetime import datetime              # тип "дата и время" - для created_at и updated_at
from typing import Annotated               # "тип + дополнительные правила"

from pydantic import BaseModel, ConfigDict, Field, StringConstraints

# строка 12: своё имя для типа "имя исполнителя"
ArtistName = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=100)]
                                           # строка; пробелы по краям срезать; длина от 1 до 100

SPOTIFY_ID_PATTERN = r"^[0-9A-Za-z]{22}$"  # регулярное выражение: ровно 22 символа, цифры и латиница
```

ItemCreate: то, что присылает клиент (строка 18)

```
class ItemCreate(BaseModel):               # наследник BaseModel = "модель Pydantic"
    """Track sent by the client: every field except id and timestamps."""
                                           # описание; попадёт в /docs, поэтому по-английски

    model_config = ConfigDict(             # настройки модели целиком
        extra="forbid",                    # прислали неизвестное поле (например "id") -> 422
        str_strip_whitespace=True,         # у всех строк срезать пробелы: "   " -> "" -> не пройдёт
min_length
        json_schema_extra={"examples": [{  # пример тела запроса - его покажет /docs
            "title": "Numb", "artists": ["Linkin Park"], "album": "Meteora",
            "duration_sec": 187, "year": 2003, "spotify_id": "2nLtzopw4rPReszdYBJU6h",
        }]},
    )

    title: str = Field(min_length=1, max_length=200, description="track title")
                                           # обязательное (значения по умолчанию нет), строка 1..200
    artists: list[ArtistName] = Field(min_length=1, max_length=10, description="1..10 artists")
                                           # список из 1..10 имён; каждое имя проверяется по ArtistName
    album: str = Field(default="", max_length=200, description="album title, may be empty")
                                           # необязательное: не прислали - будет пустая строка
    duration_sec: int = Field(ge=1, le=24 * 3600, description="duration in seconds")
                                           # целое; ge = "больше или равно" 1, le = "меньше или равно" 86400
    year: int | None = Field(default=None, ge=1900, le=2100, description="release year")
                                           # год или null; по умолчанию null
    spotify_id: str | None = Field(default=None, pattern=SPOTIFY_ID_PATTERN, description="...")
                                           # строка по шаблону (22 символа) или null
```

Как работает проверка. Приходит JSON. Pydantic идёт по полям сверху вниз и проверяет каждое: есть ли обязательное, тот ли тип, укладывается ли в правила. Все нарушения собираются в один список, и FastAPI сразу отвечает 422 — наша функция-обработчик в этом случае **даже не вызывается**. Пример ответа на `{"title": "", "artists": ["X"], "duration_sec": -1}`:

```
{"detail": [
  {"type": "string_too_short",   "loc": ["body", "title"],        "msg": "String should have at least 1
character", ...},
  {"type": "greater_than_equal", "loc": ["body", "duration_sec"], "msg": "Input should be greater than or
equal to 1", ...}
]}
```

`loc` — где ошибка: в теле запроса (`body`), поле `title`. Для параметров адреса будет `["query", "limit"]` или `["path", "item_id"]`.

ItemRead: то, что отдаёт сервер (строка 52)

```
class ItemRead(ItemCreate):                # наследует ВСЕ поля и правила ItemCreate...
    model_config = ConfigDict(extra="ignore")   # ...но лишнее поле в ответе - не ошибка, а пропуск

    id: int                                # + номер трека, его назначает сервер
    created_at: datetime                   # + когда трек создан
    updated_at: datetime                   # + когда последний раз менялся
```

Почему две модели, а не одна. Клиент не присылает `id` — его выдаёт сервер. Если бы модель была одна, клиент мог бы прислать свой `id` и сломать нумерацию. Поэтому вход (`ItemCreate`) и выход (`ItemRead`) разные — это прямое требование задания. `extra="ignore"` на выходе нужен для клиента: если сервер когда-нибудь начнёт отдавать новое поле, старый клиент не упадёт.

ItemPage и ErrorMessage (строки 63 и 72)

```
class ItemPage(BaseModel):                 # ответ на "дай список" и "найди"
    items: list[ItemRead]                  # треки этой страницы
    total: int                             # сколько всего подходит - на всех страницах вместе
    limit: int                             # сколько просили на страницу
    offset: int                            # сколько пропустили с начала

class ErrorMessage(BaseModel):             # только для /docs: так выглядит тело ошибки 404 и 409
    detail: str                            # {"detail": "track 5 not found"}
```

Пагинация — выдача по страницам. Представь длинный список: `limit` — сколько строк на странице, `offset` — сколько строк пропустить с начала. `limit=2, offset=0` — записи 1–2; `limit=2, offset=2` — записи 3–4. `total` нужен, чтобы клиент знал, сколько всего страниц.

4.2. Хранилище `app/repository.py`

Единственное место, где есть SQL и работа с файлом базы. Наружу оно отдаёт шесть действий: добавить, найти по id, страница, поиск, заменить, удалить. Остальной код не знает, что внутри SQLite.

Своя ошибка «такой трек уже есть» (строка 16)

```
class DuplicateTrackError(Exception):      # своя ошибка; main.py превратит её в ответ 409
    def __init__(self, spotify_id: str) -> None:
        super().__init__(f"track with spotify_id {spotify_id} already exists")   # текст ошибки
        self.spotify_id = spotify_id       # запомнить, какой id оказался занят
```

Протокол TrackRepository (строка 23)

```
class TrackRepository(Protocol):           # "что умеет хранилище" - без кода, только подписи
    def add(self, data: ItemCreate) -> ItemRead: ...                        # добавить, вернуть с id
    def get(self, item_id: int) -> ItemRead | None: ...                     # найти; None - нет такого
    def page(self, limit: int, offset: int) -> tuple[list[ItemRead], int]: ...   # страница + сколько всего
    def search(self, query: str, limit: int, offset: int) -> tuple[list[ItemRead], int]: ...
    def replace(self, item_id: int, data: ItemCreate) -> ItemRead | None: ...    # заменить; None - нет
такого
    def delete(self, item_id: int) -> bool: ...                             # True - удалили, False - не было
```

`...` вместо тела — значит «тут только описание». `main.py` знает только этот протокол. Захочешь хранить треки в JSON-файле — напишешь класс с такими же шестью методами, и `main.py` не изменится. Это требование задания «хранилище инкапсулировано отдельным слоем»; идея та же, что с плагинами в ЛР3.

Таблица в базе (строка 45)

```
CREATE TABLE IF NOT EXISTS tracks (                    -- создать таблицу, если её ещё нет
    id           INTEGER PRIMARY KEY AUTOINCREMENT,    -- номер: база выдаёт сама 1, 2, 3...
    title        TEXT    NOT NULL,                     -- название; NOT NULL = обязательно
    title_fold   TEXT    NOT NULL,                     -- то же название маленькими буквами, для поиска
    artists      TEXT    NOT NULL,                     -- список исполнителей, записанный как JSON-текст
    album        TEXT    NOT NULL,
    duration_sec INTEGER NOT NULL,
    year         INTEGER,                              -- может быть пустым (NULL)
    spotify_id   TEXT    UNIQUE,                       -- UNIQUE: два одинаковых не пустит
    created_at   TEXT    NOT NULL,                     -- дата текстом: "2026-10-18T14:37:41+00:00"
    updated_at   TEXT    NOT NULL
)
```

Таблица — как лист Excel: столбцы — поля, строки — треки. Почему так:

- **AUTOINCREMENT** — номер удалённого трека больше никому не выдаётся. Удалил трек 3 — адрес `/items/3` навсегда отвечает 404, а не показывает чужой трек.
- **title_fold** — встроенные в SQLite `lower()` и `LIKE` умеют менять регистр только у латиницы: «Кино» и «кино» для них разные. Поэтому маленькие буквы считает Python (`casefold()`) и кладёт в отдельный столбец.
- **artists как текст** — в SQLite нет типа «список», поэтому `["Kino"]` хранится строкой.
- **spotify_id UNIQUE** — повторный импорт того же файла не создаст дубли; пустые (NULL) друг другу не мешают.

Мелкие помощники (строки 60–92)

```
_COLUMNS = "id, title, artists, album, duration_sec, year, spotify_id, created_at, updated_at"
                                        # какие столбцы читать (title_fold - служебный, наружу не нужен)

def _now() -> str:
    return datetime.now(UTC).isoformat()   # текущее время по UTC текстом

def _to_item(row: sqlite3.Row) -> ItemRead:          # строка из базы -> объект ItemRead
    return ItemRead(
        id=row["id"],                      # столбец по имени (это даёт row_factory, см. _connect)
        title=row["title"],
        artists=json.loads(row["artists"]),           # текст '["Kino"]' -> список ["Kino"]
        album=row["album"],
        duration_sec=row["duration_sec"],
        year=row["year"],
        spotify_id=row["spotify_id"],
        created_at=datetime.fromisoformat(row["created_at"]),   # текст -> дата
        updated_at=datetime.fromisoformat(row["updated_at"]),
    )

def _fields(data: ItemCreate) -> tuple[str, str, str, str, int, int | None, str | None]:
                                        # ItemCreate -> 7 значений в порядке столбцов таблицы
    return (
        data.title,
        data.title.casefold(),             # для title_fold: "Группа крови" -> "группа крови"
        json.dumps(data.artists, ensure_ascii=False),   # список -> текст; кириллица в файле как есть
        data.album,
        data.duration_sec,
        data.year,
        data.spotify_id,
    )
```

Класс SqliteTrackRepository (строка 95)

```
class SqliteTrackRepository:                # настоящее хранилище; под протокол подходит просто потому,
                                            # что у него те же шесть методов
    def __init__(self, path: Path) -> None:      # path - где лежит файл базы
        self._path = path                   # запомнить путь
        path.parent.mkdir(parents=True, exist_ok=True)   # создать папку data/, если её нет
        with self._connect() as conn:       # открыть базу...
            conn.execute(_SCHEMA)           # ...и создать таблицу, если её ещё нет

    @property
    def path(self) -> Path:                 # прочитать путь снаружи: repo.path
        return self._path

    @contextmanager                         # превращает функцию в то, что можно писать в "with"
    def _connect(self) -> Iterator[sqlite3.Connection]:
        conn = sqlite3.connect(self._path) # открыть файл базы; файла нет - создастся пустой
        conn.row_factory = sqlite3.Row      # строки результата можно читать по имени: row["title"]
        try:
            with conn:                      # транзакция: всё прошло - сохранить (commit),
                                            # была ошибка - отменить всё целиком (rollback)
                yield conn                  # здесь выполняется код внутри "with" у того, кто позвал
        finally:
            conn.close()                    # закрыть файл в любом случае
```

Почему новое соединение на каждое действие. Обычные функции-обработчики (`def`, не `async def`) FastAPI вызывает в разных потоках, а одно соединение sqlite3 нельзя делить между потоками. Открыть файл заново — дёшево.

add: добавить трек (строка 118)

```
    def add(self, data: ItemCreate) -> ItemRead:
        now = _now()                        # одно время и для created_at, и для updated_at
        try:
            with self._connect() as conn:
                cursor = conn.execute(
                    "INSERT INTO tracks (title, title_fold, artists, album, duration_sec, year,"
                    " spotify_id, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
                    (*_fields(data), now, now),   # 7 полей трека + 2 даты = 9 значений на 9 знаков "?"
                )
                row = conn.execute(
                    f"SELECT {_COLUMNS} FROM tracks WHERE id = ?", (cursor.lastrowid,)
                ).fetchone()                # lastrowid - id, который только что выдала база
        except sqlite3.IntegrityError as exc:     # база отказала: UNIQUE нарушен, spotify_id занят
            raise DuplicateTrackError(data.spotify_id or "") from exc   # перевести в нашу ошибку
        return _to_item(row)                # строка базы -> ItemRead
```

> **Знаки `?` в SQL.** Значения передаются отдельно от текста запроса, и sqlite3 сам их подставляет безопасно. Если вставлять пользовательский текст в SQL через f-строку, название вида `'); DROP TABLE tracks; --` выполнилось бы как команда (SQL-инъекция). В f-строку у нас попадает только наша константа `_COLUMNS` — это безопасно.

get, page, search (строки 134–160)

```
    def get(self, item_id: int) -> ItemRead | None:
        with self._connect() as conn:
            row = conn.execute(f"SELECT {_COLUMNS} FROM tracks WHERE id = ?", (item_id,)).fetchone()
                                            # fetchone(): одна строка или None, если такой нет
        return _to_item(row) if row else None

    def page(self, limit: int, offset: int) -> tuple[list[ItemRead], int]:
        with self._connect() as conn:
```

```
            rows = conn.execute(
                f"SELECT {_COLUMNS} FROM tracks ORDER BY id LIMIT ? OFFSET ?", (limit, offset)
            ).fetchall()                    # по порядку id; взять limit штук, пропустив offset
            total = conn.execute("SELECT COUNT(*) FROM tracks").fetchone()[0]   # сколько всего
        return [_to_item(row) for row in rows], int(total)   # (список треков, общее число)

    def search(self, query: str, limit: int, offset: int) -> tuple[list[ItemRead], int]:
        needle = query.casefold()           # что ищем, маленькими буквами: "ГРУППА" -> "группа"
        with self._connect() as conn:
            rows = conn.execute(
                f"SELECT {_COLUMNS} FROM tracks WHERE instr(title_fold, ?) > 0"
                " ORDER BY id LIMIT ? OFFSET ?",
                (needle, limit, offset),   # instr = номер позиции подстроки, 0 - не нашлась
            ).fetchall()
            total = conn.execute(
                "SELECT COUNT(*) FROM tracks WHERE instr(title_fold, ?) > 0", (needle,)
            ).fetchone()[0]                 # сколько всего нашлось - для total
        return [_to_item(row) for row in rows], int(total)
```

Почему `instr`, а не `LIKE`: в `LIKE` символы `%` и `_` — шаблоны («что угодно» и «один любой символ»). Поиск «100%» через LIKE нашёл бы всё, что начинается на «100». `instr` ищет текст буквально.

replace и delete (строки 162–182)

```
    def replace(self, item_id: int, data: ItemCreate) -> ItemRead | None:
        try:
            with self._connect() as conn:
                cursor = conn.execute(
                    "UPDATE tracks SET title = ?, title_fold = ?, artists = ?, album = ?,"
                    " duration_sec = ?, year = ?, spotify_id = ?, updated_at = ? WHERE id = ?",
                    (*_fields(data), _now(), item_id),   # все поля новые, updated_at = сейчас
                )                           # created_at не трогаем: дата создания остаётся прежней
                if cursor.rowcount == 0:   # rowcount - сколько строк изменилось; 0 = такого id нет
                    return None
                row = conn.execute(
                    f"SELECT {_COLUMNS} FROM tracks WHERE id = ?", (item_id,)
                ).fetchone()                # прочитать трек уже после изменения
        except sqlite3.IntegrityError as exc:     # поставили spotify_id, который занят другим треком
            raise DuplicateTrackError(data.spotify_id or "") from exc
        return _to_item(row)

    def delete(self, item_id: int) -> bool:
        with self._connect() as conn:
            cursor = conn.execute("DELETE FROM tracks WHERE id = ?", (item_id,))
        return cursor.rowcount > 0          # удалилась строка - True, такого id не было - False
```

4.3. Адреса API `app/main.py`

Главный файл сервера. Здесь создаётся приложение и объявлены шесть адресов. Каждый адрес — обычная функция с декоратором; FastAPI сам достаёт для неё данные из запроса и сам упаковывает ответ.

Импорты и путь к базе (строки 5–20)

```
import json                                  # упаковка в JSON
import os                                    # переменные окружения
from functools import lru_cache              # "запомнить результат функции"
from pathlib import Path                     # пути к файлам
from typing import Annotated, Any

from fastapi import Depends, FastAPI, HTTPException, Query, Request, Response, status
from fastapi.encoders import jsonable_encoder         # объекты -> то, что можно положить в JSON
from fastapi.exceptions import RequestValidationError # ошибка "запрос не прошёл проверку"
from fastapi.responses import JSONResponse, RedirectResponse

from app.repository import DuplicateTrackError, SqliteTrackRepository, TrackRepository
from app.schemas import ErrorMessage, ItemCreate, ItemPage, ItemRead

DEFAULT_DB = Path("data") / "tracks.db"    # рабочая база: data/tracks.db рядом с проектом
```

Ответы только в ASCII (строка 23)

```
class AsciiJSONResponse(JSONResponse):     # наш вид ответа: обычный JSON-ответ...
    def render(self, content: Any) -> bytes:      # ...со своей "упаковкой в байты"
        return json.dumps(content, ensure_ascii=True, allow_nan=False).encode("ascii")
                                           # ensure_ascii: "Кино" -> "\u041a\u0438\u043d\u043e"
                                           # allow_nan=False: NaN - не JSON, лучше честная ошибка
```

Задание требует, чтобы весь вывод программы был ASCII. Запись `\u041a\u0438\u043d\u043e` — законный JSON: клиент после разбора (`response.json()`) получает обратно настоящее «Кино». Без этого класса FastAPI отдавал бы кириллицу в UTF-8.

Где взять хранилище (строка 31)

```
@lru_cache(maxsize=1)                      # запомнить результат: второй вызов вернёт тот же объект
def get_repository() -> TrackRepository:
    return SqliteTrackRepository(Path(os.environ.get("TRACKS_DB", DEFAULT_DB)))
                                           # путь из переменной окружения TRACKS_DB, иначе data/tracks.db
```

Хранилище создаётся **при первом запросе**, а не при запуске, и дальше всё время одно и то же. Тесты подменяют именно эту функцию — поэтому рабочая база в тестах ни разу не открывается.

Сокращения для аргументов (строки 38–47)

```
Repo = Annotated[TrackRepository, Depends(get_repository)]
                                           # "аргумент-хранилище: перед вызовом возьми из get_repository()"
Limit = Annotated[int, Query(ge=1, le=100, description="page size, 1..100")]
                                           # "целое из ?limit=, от 1 до 100"
Offset = Annotated[int, Query(ge=0, description="how many tracks to skip")]
                                           # "целое из ?offset=, не меньше 0"

NOT_FOUND: dict[int | str, dict[str, Any]] = {
    404: {"model": ErrorMessage, "description": "Track not found"}
}                                          # только для /docs: "у этого адреса бывает ещё 404"
CONFLICT: dict[int | str, dict[str, Any]] = {
    409: {"model": ErrorMessage, "description": "spotify_id already used by another track"}
}                                          # и "бывает 409"
```

Само приложение и ответ 422 (строки 49–64)

```
app = FastAPI(
    title="Music Library API",             # заголовок страницы /docs
    version="1.0.0",
    summary="Lab 4: REST service over music tracks (FastAPI + Pydantic + SQLite).",
    default_response_class=AsciiJSONResponse,     # все ответы упаковывать нашим классом
)

@app.exception_handler(RequestValidationError)    # "когда запрос не прошёл проверку - вызывай меня"
async def validation_error(request: Request, exc: RequestValidationError) -> AsciiJSONResponse:
    return AsciiJSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_CONTENT,   # код 422
        content={"detail": jsonable_encoder(exc.errors())},  # тот же список ошибок, что у FastAPI
    )                                      # по умолчанию, только тоже в ASCII
```

Помощники и корень (строки 67–77)

```
def _not_found(item_id: int) -> HTTPException:
    return HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"track {item_id} not found")
                                           # HTTPException - особая ошибка: FastAPI ловит её сам и
                                           # отвечает этим кодом и телом {"detail": "..."}

def _conflict(exc: DuplicateTrackError) -> HTTPException:
    return HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc))

@app.get("/", include_in_schema=False)     # открыли в браузере просто адрес сервера...
def root() -> RedirectResponse:
    return RedirectResponse("/docs")       # ...перекинуть на /docs; в самой документации не показывать
```

POST /items — добавить трек (строка 80)

```
@app.post(                                 # "эта функция - для POST-запросов..."
    "/items",                              # "...на адрес /items"
    response_model=ItemRead,               # ответ упаковать строго по ItemRead (лишнее отбросить)
    status_code=status.HTTP_201_CREATED,   # успех = 201 Created, а не 200
    responses=CONFLICT,                    # для /docs: ещё бывает 409
    tags=["items"],                        # группа на странице /docs
)
def create_item(data: ItemCreate, repo: Repo, response: Response) -> ItemRead:
                                           # data - тело запроса, УЖЕ проверенное по ItemCreate
                                           # repo - хранилище, его подставил FastAPI через Depends
                                           # response - чтобы добавить заголовок к ответу
    """Add a track. Returns 201 with the stored track, its new id and a Location header."""
    try:
        item = repo.add(data)              # сохранить; получить трек уже с id
    except DuplicateTrackError as exc:     # spotify_id занят
        raise _conflict(exc) from exc      # -> ответ 409
    response.headers["Location"] = f"/items/{item.id}"   # обычай REST: где теперь лежит созданное
    return item                            # FastAPI превратит в JSON и ответит с кодом 201
```

> **Откуда FastAPI знает, где брать каждый аргумент.** По типу и имени: модель Pydantic (`ItemCreate`) — из тела запроса; `Annotated[..., Depends(...)]` — вызвать функцию; `Response` — особый объект ответа; имя есть в адресе (`{item_id}`) — из адреса; простое `int` / `str`, которого нет в адресе, — из query (`?limit=20`).

GET /items — список по страницам (строка 97)

```
@app.get("/items", response_model=ItemPage, tags=["items"])
def list_items(repo: Repo, limit: Limit = 20, offset: Offset = 0) -> ItemPage:
                                        # limit и offset из адреса; не указаны - 20 и 0
    items, total = repo.page(limit, offset)       # хранилище вернуло (треки, сколько всего)
    return ItemPage(items=items, total=total, limit=limit, offset=offset)   # упаковать в страницу
```

GET /items/search — поиск (строка 106)

```
# Порядок важен: /items/search объявлен раньше /items/{item_id}
@app.get("/items/search", response_model=ItemPage, tags=["items"])
def search_items(
    repo: Repo,
    q: Annotated[str, Query(min_length=1, max_length=100, description="part of the title")],
                                        # q обязательный (нет значения по умолчанию), 1..100 символов
    limit: Limit = 20,
    offset: Offset = 0,
) -> ItemPage:
    items, total = repo.search(q, limit, offset)
    return ItemPage(items=items, total=total, limit=limit, offset=offset)
```

> **Почему важен порядок.** FastAPI проверяет адреса в том порядке, в каком они объявлены. Шаблону `/items/{item_id}` подходит *любое* слово после `/items/`, в том числе `search`. Если бы он стоял выше, запрос `/items/search?q=numb` попал бы в `get_item` с `item_id="search"`, и FastAPI ответил бы 422 «это не целое число».

GET, PUT, DELETE /items/{item_id} (строки 118–154)

```
@app.get("/items/{item_id}", response_model=ItemRead, responses=NOT_FOUND, tags=["items"])
def get_item(item_id: int, repo: Repo) -> ItemRead:      # item_id из адреса; не число -> 422
    item = repo.get(item_id)
    if item is None:                        # такого трека нет...
        raise _not_found(item_id)           # ...ответ 404, а не падение с 500
    return item                             # есть - 200 и трек

@app.put("/items/{item_id}", response_model=ItemRead, responses={**NOT_FOUND, **CONFLICT}, tags=["items"])
def replace_item(item_id: int, data: ItemCreate, repo: Repo) -> ItemRead:
                                        # PUT = заменить все поля целиком; тело - снова ItemCreate
    try:
        item = repo.replace(item_id, data)
    except DuplicateTrackError as exc:      # новый spotify_id занят другим треком
        raise _conflict(exc) from exc       # -> 409
    if item is None:                        # такого трека нет
        raise _not_found(item_id)           # -> 404
    return item                             # -> 200 и обновлённый трек

@app.delete(
    "/items/{item_id}",
    status_code=status.HTTP_204_NO_CONTENT,       # успех = 204: "сделано, показывать нечего"
    response_class=Response,                # пустой ответ, без JSON
    responses=NOT_FOUND,
    tags=["items"],
)
def delete_item(item_id: int, repo: Repo) -> None:
    if not repo.delete(item_id):            # удалять было нечего
        raise _not_found(item_id)           # -> 404
                                        # удалили - функция просто заканчивается -> 204
```

Этих шести функций хватает на весь CRUD (Create, Read, Update, Delete) плюс поиск. Валидацию, разбор JSON, коды 422 и документацию `/docs` FastAPI делает сам — вручную мы пишем только 404 и 409.

4.4. Перевод в ASCII `client/text.py`

Перенесён из ЛР3 без изменений. Задание требует, чтобы программа печатала только ASCII, поэтому перед печатью клиент пропускает каждую строку через `to_ascii()`.

```
_LATIN = ["a", "b", "v", "g", "d", "e", "zh", "z", "i", "y", "k", "l", "m", "n", "o", "p",
          "r", "s", "t", "u", "f", "kh", "ts", "ch", "sh", "shch", "", "y", "", "e", "yu", "ya"]
                                          # латиница для а, б, в ... я - строго по алфавиту
CYRILLIC = {chr(0x0430 + i): latin for i, latin in enumerate(_LATIN)}
                                          # 0x0430 - код буквы "а"; дальше буквы идут подряд,
                                          # поэтому получаем {"а": "a", "б": "b", ..., "я": "ya"}
CYRILLIC |= {"ё": "e", "і": "i", "ї": "yi", "є": "ye", "ґ": "g"}
                                          # добавить ё, і, ї, є, ґ - они стоят отдельно от алфавита
PUNCTUATION = {"—": "-", "«": '"', "…": "...", ...}
                                          # длинное тире, кавычки-ёлочки, многоточие -> ASCII

def to_ascii(text: str) -> str:
    out: list[str] = []                   # сюда складываем готовые кусочки
    for ch in text:                       # идём по одному символу
        if ch.isascii():                  # обычная латиница, цифры, знаки -
            out.append(ch)                #   как есть
        elif ch in PUNCTUATION:           # типографский знак -
            out.append(PUNCTUATION[ch])   #   его ASCII-замена
        elif ch.lower() in CYRILLIC:      # кириллица -
            latin = CYRILLIC[ch.lower()]  #   латиница для маленькой буквы
            out.append(latin if ch.islower() else latin.capitalize())   # "Г" -> "G", "ж" -> "zh"
        else:                             # всё остальное (é, Ÿ, иероглифы)
            base = unicodedata.normalize("NFKD", ch).encode("ascii", "ignore").decode()
                                          # NFKD раскладывает "é" на "e" + значок; значок выбрасываем
            out.append(base or ch.encode("ascii", "backslashreplace").decode())
                                          # не разложилось (иероглиф) - пишем \uXXXX
    return "".join(out)                   # склеить кусочки: "Группа крови" -> "Gruppa krovi"
```

4.5. Клиент на httpx `client/api.py`

Класс `LibraryClient` — «пульт» от сервера: у него по методу на каждое действие (`create`, `get`, `list_items`, `search`, `replace`, `delete`). Внутри он отправляет HTTP-запросы через httpx и делает три обязательные по заданию вещи: **таймаут**, **проверку кода ответа** и **понятные ошибки**. Ещё он возвращает не голый JSON, а типизированные объекты `ItemRead` и `ItemPage`.

Свои ошибки (строки 17–45)

```
DEFAULT_URL = "http://127.0.0.1:8000"      # куда стучаться, если --url не указан
DEFAULT_TIMEOUT = 5.0                      # ждать ответа не дольше 5 секунд

class ApiError(Exception): ...             # общая ошибка клиента; текст всегда ASCII
class NotFoundError(ApiError): ...         # 404 - такого трека нет
class InvalidInputError(ApiError): ...     # 422 - неправильные данные
class ConflictError(ApiError): ...         # 409 - spotify_id уже занят
class ServerError(ApiError): ...           # 5xx - сломался сервер (и после всех повторов)
class ServiceUnavailableError(ApiError): ...   # ответа нет вообще: сервер выключен, таймаут
```

Все ошибки — наследники `ApiError`. Командная строка ловит одну `ApiError` и печатает текст; а импорт, например, ловит именно `ConflictError`, чтобы пропустить уже добавленный трек.

Достать причину ошибки из ответа (строка 48)

```
def _error_detail(response: httpx.Response) -> str:
    try:
        detail = response.json()["detail"] # FastAPI кладёт причину ошибки в поле "detail"
    except (ValueError, KeyError, TypeError):    # ответ не JSON / нет "detail" / не словарь
```

```
        return to_ascii(response.text[:200]) or response.reason_phrase   # показать начало ответа как есть
    if isinstance(detail, list):          # 422: detail - список ошибок по полям
        parts = []
        for error in detail:
            field = ".".join(str(part) for part in error.get("loc", [])[1:]) or "request"
                                          # ["body", "title"] -> "title"; ["query", "limit"] -> "limit"
            parts.append(f"{field}: {error.get('msg', 'invalid')}")   # "title: String should have..."
        return to_ascii("; ".join(parts))  # все ошибки через "; "
    return to_ascii(str(detail))          # 404 / 409: просто строка "track 5 not found"
```

Код ответа → наша ошибка (строка 64)

```
def _status_error(exc: httpx.HTTPStatusError) -> ApiError:
    response = exc.response
    code = response.status_code
    detail = _error_detail(response)
    if code == 404:
        return NotFoundError(f"not found (404): {detail}")
    if code == 409:
        return ConflictError(f"conflict (409): {detail}")
    if code == 422:
        return InvalidInputError(f"invalid input (422): {detail}")
    if code >= 500:                       # 500, 502, 503...
        return ServerError(f"server error ({code}): {detail}")
    return ApiError(f"unexpected answer {code} {response.reason_phrase}: {detail}")   # всё прочее
```

Создание клиента (строки 79–107)

```
class LibraryClient:
    def __init__(
        self,
        base_url: str = DEFAULT_URL,      # адрес сервера
        *,                                # всё, что дальше, - только по имени: timeout=3
        timeout: float = DEFAULT_TIMEOUT,  # сколько ждать каждый запрос
        retries: int = 2,                 # сколько раз повторить после сбоя (итого до 3 попыток)
        backoff: float = 0.5,             # первая пауза между попытками, дальше вдвое больше
        transport: httpx.BaseTransport | None = None,   # подмена "сети" - только для тестов
    ) -> None:
        self._http = httpx.Client(base_url=base_url, timeout=timeout, transport=transport)
                                          # один httpx-клиент на все запросы; base_url
                                          # подставляется сам: "/items" -> "http://.../items"
        self._timeout = timeout
        self._retries = retries
        self._backoff = backoff

    def __enter__(self) -> Self:          # "with LibraryClient(...) as client:" ...
        return self
    def __exit__(self, exc_type, exc, tb) -> None:   # ... выход из with, даже по ошибке ->
        self.close()                      # закрыть соединения
    def close(self) -> None:
        self._http.close()
```

Шесть действий (строки 111–131)

```
    def create(self, data: ItemCreate) -> ItemRead:
        response = self._request("POST", "/items", json=data.model_dump(mode="json"), retry=False)
                                          # model_dump: модель -> словарь для JSON
                                          # retry=False: POST не повторяем, иначе может быть дубль
        return self._parse(ItemRead, response)    # JSON ответа -> ItemRead

    def get(self, item_id: int) -> ItemRead:
        return self._parse(ItemRead, self._request("GET", f"/items/{item_id}"))

    def list_items(self, limit: int = 20, offset: int = 0) -> ItemPage:
```

```
        response = self._request("GET", "/items", params={"limit": limit, "offset": offset})
                                        # params -> в адрес: /items?limit=20&offset=0
        return self._parse(ItemPage, response)

    def search(self, query: str, limit: int = 20, offset: int = 0) -> ItemPage:
        params: dict[str, str | int] = {"q": query, "limit": limit, "offset": offset}
        return self._parse(ItemPage, self._request("GET", "/items/search", params=params))
                                        # кириллицу в адресе httpx закодирует сам: %D0%B3...

    def replace(self, item_id: int, data: ItemCreate) -> ItemRead:
        response = self._request("PUT", f"/items/{item_id}", json=data.model_dump(mode="json"))
        return self._parse(ItemRead, response)

    def delete(self, item_id: int) -> None:
        self._request("DELETE", f"/items/{item_id}")   # ответ 204 пустой - возвращать нечего
```

_request: сердце клиента (строка 135)

```
    def _request(self, method, url, *, json=None, params=None, retry=True) -> httpx.Response:
        attempts = 1 + (self._retries if retry else 0)   # GET/PUT/DELETE: 1 + 2 = 3 попытки; POST: 1
        for attempt in range(1, attempts + 1):     # attempt = 1, 2, 3
            last_try = attempt == attempts         # это последняя попытка?
            try:
                response = self._http.request(method, url, json=json, params=params)
                                        # отправить запрос и ждать ответа - не дольше timeout
                response.raise_for_status()        # код 4xx или 5xx -> исключение HTTPStatusError
                return response                    # 2xx: всё хорошо, выходим из функции
            except httpx.HTTPStatusError as exc:   # сервер ответил, но ошибкой
                if exc.response.status_code < 500 or last_try:
                                        # 4xx (виноват запрос - повтор бесполезен)
                                        # или попытки кончились
                    raise _status_error(exc) from exc   # -> наша понятная ошибка
                                        # 5xx и попытки ещё есть: идём вниз, к паузе
            except httpx.RequestError as exc:      # ответа нет совсем: таймаут, нет соединения, обрыв
                if last_try:
                    raise ServiceUnavailableError(self._network_message(exc)) from exc
            time.sleep(self._backoff * 2 ** (attempt - 1))   # пауза: 0.5 с, потом 1 с
        raise AssertionError("unreachable: the last attempt always returns or raises")
                                        # сюда не дойдём: последняя попытка либо вернула ответ,
                                        # либо бросила ошибку; строка нужна для муру
```

Пример: GET, сервер дважды отвечает 503, потом 200. Попытка 1 → 503 → пауза 0,5 с → попытка 2 → 503 → пауза 1 с → попытка 3 → 200 → `return`. Если бы это был POST — первая же 503 сразу стала бы `ServerError`.

> **Зачем `raise_for_status()`.** httpx по умолчанию считает ответ 404 или 500 «нормально полученным» и исключение не бросает. Без этой строки клиент пошёл бы дальше и упал бы где-то глубже, на разборе чужого JSON. **Зачем `from exc`.** Исходная ошибка httpx сохраняется как «причина» — пригодится при отладке, а пользователю показывается только наш текст.

Текст сетевой ошибки и разбор ответа (строки 163–180)

```
    def _network_message(self, exc: httpx.RequestError) -> str:
        where = str(self._http.base_url).rstrip("/")   # "http://127.0.0.1:8000"
        kind = type(exc).__name__          # имя класса ошибки: "ConnectError", "ReadTimeout"
        if isinstance(exc, httpx.TimeoutException):    # сервер принял соединение, но молчит
            return f"{where} did not answer within {self._timeout:g} s ({kind})"
        if isinstance(exc, httpx.ConnectError):        # подключиться не удалось вообще
            return f"cannot connect to {where} ({kind}); is the server running?"
        return f"network error while talking to {where} ({kind})"   # обрыв и прочее

    @staticmethod                          # метод без self: ему не нужен сам клиент
```

```
    def _parse(model: type[M], response: httpx.Response) -> M:
                                           # M - "какой класс передали, такой и вернётся":
                                           # _parse(ItemRead, ...) -> ItemRead (муру это знает)
        try:
            return model.model_validate(response.json())   # JSON -> модель, с проверкой полей
        except ValueError as exc:          # не JSON или не те поля
            raise ApiError(
                f"unexpected response body for {response.request.method} {response.url.path}"
            ) from exc
```

`_parse` — защита от сюрпризов: если по адресу отвечает не наш сервер (например, страница провайдера или прокси), клиент скажет «unexpected response body», а не упадёт с непонятным трейсбеком.

4.6. Командная строка `client/cli.py`

Превращает команду вида `python -m client add --title ...` в вызов метода `LibraryClient` и печатает результат одной строкой.

Мелкие помощники (строки 20–61)

```
def _force_ascii(stream: object) -> None:
    if isinstance(stream, io.TextIOWrapper):      # обычный вывод в консоль
        stream.reconfigure(encoding="ascii", errors="backslashreplace")
                                          # не-ASCII символ, если всё же дошёл до печати, станет \uXXXX

def _duration(text: str) -> int:          # "3:07" или "187" -> секунды
    minutes, _, seconds = text.rpartition(":")    # "3:07" -> ("3", ":", "07"); "187" -> ("", "", "187")
    try:
        return int(minutes or 0) * 60 + int(seconds)   # 3*60 + 7 = 187; 0*60 + 187 = 187
    except ValueError:                    # "abc" - не число
        raise argparse.ArgumentTypeError(f"expected M:SS or seconds, got {ascii(text)}") from None

def _clock(seconds: int) -> str:          # 187 -> "3:07"
    minutes, rest = divmod(seconds, 60)    # divmod(187, 60) = (3, 7): целая часть и остаток
    return f"{minutes}:{rest:02d}"        # :02d - две цифры с нулём впереди: 7 -> "07"

def _line(item: ItemRead) -> str:         # одна строка вывода про трек
    extra = ", ".join(part for part in (item.album, str(item.year or "")) if part)
                                          # "Meteora, 2003"; пустые части пропускаются
    text = f"#{item.id:<4} {', '.join(item.artists)} - {item.title}"
                                          # :<4 - id прижат влево и дополнен пробелами до 4 символов
    if extra:
        text += f" [{extra}]"
    return to_ascii(f"{text} {_clock(item.duration_sec)}")
                                          # "#3    Linkin Park - Numb [Meteora, 2003] 3:07"

def _print_page(page: ItemPage) -> None:   # напечатать страницу списка или поиска
    if not page.items:
        print("no tracks" if page.total == 0 else f"no tracks on this page (total {page.total})")
        return
    first = page.offset + 1               # нумерация для человека - с 1
    print(f"tracks {first}-{page.offset + len(page.items)} of {page.total}:")   # "tracks 1-3 of 4:"
    for item in page.items:
        print(_line(item))

def _validation_message(exc: ValidationError) -> str:   # ошибка Pydantic -> одна строка
    parts = [f"{'.'.join(str(p) for p in e['loc']) or 'track'}: {e['msg']}" for e in exc.errors()]
    return to_ascii("invalid input: " + "; ".join(parts))   # "invalid input: title: String should..."
```

Трек из аргументов (строка 64)

```
def _track_from_args(args: argparse.Namespace, current: ItemRead | None = None) -> ItemCreate:
    data = current.model_dump(include=set(ItemCreate.model_fields)) if current else {}
                                          # update: начинаем с текущего трека (без id и дат);
                                          # add: с пустого словаря
    changes = {
        "title": args.title,              # то, что указали в командной строке;
        "artists": args.artist,           # не указанное равно None
        "album": args.album,
        "duration_sec": args.duration,
        "year": args.year,
        "spotify_id": args.spotify_id,
    }
    data.update({key: value for key, value in changes.items() if value is not None})
                                          # наложить только указанные поля
    return ItemCreate.model_validate(data) # проверить той же моделью, что и сервер
```

Так `update 5 --year 1991` меняет только год: клиент берёт трек с сервера (GET), меняет одно поле и отправляет целиком (PUT). PUT по правилам REST заменяет трек полностью, поэтому отправлять нужно все поля.

Импорт JSON из ЛР3 (строки 80–117)

```
def _load_lab3_json(path: Path) -> list[ItemCreate]:
    data = json.loads(path.read_text(encoding="utf-8"))   # прочитать файл и разобрать JSON
    if not isinstance(data, list):         # в файле должен быть список треков
        raise ValueError(f"{path.name}: expected a JSON array of tracks")
    tracks = []
    for index, raw in enumerate(data):     # index - номер трека в файле, для текста ошибки
        try:
            tracks.append(ItemCreate(
                title=raw["title"],
                artists=raw["artists"],
                album=raw.get("album", ""),           # нет альбома - пустая строка
                duration_sec=max(1, int(raw["duration_ms"]) // 1000),
                                       # в ЛР3 миллисекунды; // 1000 - отбросить доли секунды,
                                       # как в ЛР3 и в приложении Spotify (187733 мс -> 187 с)
                spotify_id=raw.get("id"),  # id из Spotify -> наш spotify_id
            ))
        except ValidationError as exc:     # поле есть, но значение не подходит
            raise ValueError(f"{path.name}, track #{index}: {_validation_message(exc)}") from exc
        except (KeyError, TypeError) as exc:      # поля нет / элемент не словарь
            raise ValueError(f"{path.name}, track #{index}: missing or bad field {exc}") from exc
    return tracks

def _import(client: LibraryClient, path: Path) -> None:
    created = skipped = 0                  # два счётчика
    for track in _load_lab3_json(path):
        try:
            item = client.create(track)    # POST /items
        except ConflictError:              # 409: трек с таким spotify_id уже есть
            skipped += 1
            print(to_ascii(f"skip: {', '.join(track.artists)} - {track.title} (already in library)"))
            continue                       # к следующему треку
        created += 1
        print(f"added {_line(item)}")
    print(f"imported {created} tracks, skipped {skipped}")
```

Повторный импорт того же файла ничего не дублирует: сервер отвечает 409, клиент пишет «skip». Это и есть идемпотентность «на уровне смысла»: повтор безопасен благодаря уникальному ключу `spotify_id`.

Описание команд (строки 120–157)

```
def _add_track_options(parser, required: bool) -> None:   # общие флаги для add и update
    parser.add_argument("--title", required=required)
    parser.add_argument("--artist", action="append", required=required, help="repeat for several artists")
                                       # append: --artist A --artist B -> ["A", "B"]
    parser.add_argument("--duration", type=_duration, required=required, help="M:SS or seconds")
                                       # type=_duration: argparse сам вызовет нашу функцию
    parser.add_argument("--album")
    parser.add_argument("--year", type=int)
    parser.add_argument("--spotify-id")    # в коде будет args.spotify_id

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="python -m client", description="Client for the Music Library REST
API.")
    parser.add_argument("--url", default=DEFAULT_URL, ...)        # общие флаги - до команды
    parser.add_argument("--timeout", type=float, default=DEFAULT_TIMEOUT, ...)
    commands = parser.add_subparsers(dest="command", required=True)   # команда обязательна

    for name, help_text in (("list", "..."), ("search", "...")):  # у list и search одинаковые флаги
```

```
        sub = commands.add_parser(name, help=help_text)
        if name == "search":
            sub.add_argument("query")      # у search ещё сам текст поиска
        sub.add_argument("--limit", type=int, default=20)
        sub.add_argument("--offset", type=int, default=0)

    commands.add_parser("get", help="show one track").add_argument("id", type=int)
    _add_track_options(commands.add_parser("add", help="add a track"), required=True)
                                       # для add title/artist/duration обязательны
    update = commands.add_parser("update", help="change fields of a track (PUT)")
    update.add_argument("id", type=int)
    _add_track_options(update, required=False)    # для update - все флаги по желанию
    commands.add_parser("delete", help="delete a track").add_argument("id", type=int)
    commands.add_parser("import", help="...").add_argument("file", type=Path)
    return parser
```

argparse бесплатно даёт подсказку: `python -m client --help`, `python -m client add --help`.

Выполнить команду и main (строки 160–196)

```
def run(client: LibraryClient, args: argparse.Namespace) -> None:
    match args.command:                    # выбрать действие по имени команды
        case "list":
            _print_page(client.list_items(args.limit, args.offset))
        case "search":
            _print_page(client.search(args.query, args.limit, args.offset))
        case "get":
            print(_line(client.get(args.id)))
        case "add":
            print(f"created {_line(client.create(_track_from_args(args)))}")
        case "update":
            track = _track_from_args(args, current=client.get(args.id))   # GET текущего + правки
            print(f"updated {_line(client.replace(args.id, track))}")     # PUT
        case "delete":
            client.delete(args.id)
            print(f"deleted #{args.id}")
        case "import":
            _import(client, args.file)

def main(argv: Sequence[str] | None = None) -> int:   # argv - для тестов; обычно None = sys.argv
    _force_ascii(sys.stdout)
    _force_ascii(sys.stderr)
    args = build_parser().parse_args(argv) # разобрать аргументы; ошибка в них - argparse выйдет с кодом 2
    try:
        with LibraryClient(args.url, timeout=args.timeout) as client:   # создать клиент, в конце закрыть
            run(client, args)
    except ApiError as exc:                # 404, 422, 409, 5xx, нет сети
        print(f"error: {exc}", file=sys.stderr)
        return 1
    except ValidationError as exc:         # плохой ввод, пойманный ещё до сети
        print(f"error: {_validation_message(exc)}", file=sys.stderr)
        return 1
    except (OSError, ValueError) as exc:   # нет файла для import, битый JSON
        print(f"error: {to_ascii(str(exc))}", file=sys.stderr)
        return 1
    return 0                               # всё хорошо
```

Порядок `except` важен: `ValidationError` — наследник `ValueError`. Если бы `ValueError` стоял выше, он перехватывал бы и ошибки Pydantic, и сообщение было бы хуже.

4.7. Точка входа `client/__main__.py`

```
from client.cli import main

raise SystemExit(main())                    # запустить main() и отдать её код (0 или 1) системе
```

Файл `__main__.py` Python запускает, когда пишешь `python -m client`. Код возврата виден в консоли: `echo $?` в bash, `$LASTEXITCODE` в PowerShell.

4.8. Заготовки для тестов `tests/conftest.py`

Как устроен тест pytest. Тест — функция с именем `test_...`, внутри которой есть `assert` («утверждаю, что...»). Утверждение неверно — тест красный. **Фикстура** — заготовка: если у теста есть аргумент `api`, pytest найдёт функцию-фикстуру с таким именем, вызовет её и передаст результат. Фикстуры из `conftest.py` доступны всем тестам.

```
@pytest.fixture
def repo(tmp_path: Path) -> SqliteTrackRepository:
                                        # tmp_path - встроенная фикстура: новая пустая папка на каждый
тест
    return SqliteTrackRepository(tmp_path / "tracks.db")   # своя чистая база для каждого теста

@pytest.fixture
def api(repo: SqliteTrackRepository) -> Iterator[TestClient]:
    app.dependency_overrides[get_repository] = lambda: repo
                                        # "вместо get_repository() подставляй временную базу"
    with TestClient(app) as client:     # TestClient - "клиент без сети": вызывает приложение напрямую
        yield client                    # здесь выполняется сам тест
    app.dependency_overrides.clear()    # после теста убрать подмену
```

Именно эта подмена делает тесты **изолированными**: настоящая `get_repository()` не вызывается, рабочая `data/tracks.db` не открывается. Это требование задания.

```
@pytest.fixture
def live_url(repo: SqliteTrackRepository) -> Iterator[str]:   # настоящий сервер для тестов клиента
    app.dependency_overrides[get_repository] = lambda: repo  # база - тоже временная
    sock = socket.socket()
    sock.bind(("127.0.0.1", 0))         # порт 0 = "дай любой свободный"
    port = sock.getsockname()[1]        # узнать, какой порт выдала система
    server = uvicorn.Server(uvicorn.Config(app, log_level="warning"))   # настоящий uvicorn
    thread = threading.Thread(target=server.run, kwargs={"sockets": [sock]}, daemon=True)
    thread.start()                      # запустить его в фоновом потоке
    deadline = time.monotonic() + 10
    while not server.started:           # ждать, пока поднимется (не дольше 10 с)
        if time.monotonic() > deadline or not thread.is_alive():
            raise RuntimeError("uvicorn did not start")
        time.sleep(0.02)
    yield f"http://127.0.0.1:{port}"    # тест получает адрес: "http://127.0.0.1:53817"
    server.should_exit = True           # после теста - остановить сервер
    thread.join(timeout=10)
    sock.close()
    app.dependency_overrides.clear()

@pytest.fixture
def numb() -> dict[str, Any]:           # готовый трек для тестов (латиница)
    return {"title": "Numb", "artists": ["Linkin Park"], "album": "Meteora",
            "duration_sec": 187, "year": 2003, "spotify_id": "2nLtzopw4rPReszdYBJU6h"}

@pytest.fixture
def kino() -> dict[str, Any]:           # трек с кириллицей: "Группа крови", "Кино"
    return {"title": "Группа крови", "artists": ["Кино"], "album": "Группа крови",
            "duration_sec": 284, "year": 1988}   # кириллица - для проверки поиска и ASCII-вывода
```

4.9. Тесты адресов `tests/test_api.py`

Каждый тест устроен одинаково, в три блока через пустую строку: **подготовка** → **действие** → **проверка**.

```
def test_create_returns_201_with_new_id_and_location(api: TestClient, numb: dict[str, Any]) -> None:
    response = api.post("/items", json=numb)      # действие: отправить POST с треком

    assert response.status_code == 201     # код 201
```

```
    body = response.json()
    assert body["id"] == 1                # первому треку выдан id 1
    assert {key: body[key] for key in numb} == numb   # все наши поля вернулись без изменений
    assert body["created_at"] == body["updated_at"]   # только что создан - даты совпадают
    assert response.headers["location"] == "/items/1" # заголовок Location на месте
```

Параметризация — один тест на много случаев. pytest запустит его 9 раз, по разу на каждую строку списка:

```
@pytest.mark.parametrize(
    ("field", "value"),
    [
        ("title", ""), ("title", "   "), ("title", "x" * 201),   # пустое, из пробелов, слишком длинное
        ("artists", []), ("artists", [""]),                      # нет исполнителей, пустое имя
        ("duration_sec", 0), ("duration_sec", "long"),           # ноль, не число
        ("year", 1800), ("spotify_id", "not-a-spotify-id"),      # слишком старый, не по шаблону
    ],
)
def test_create_with_invalid_field_gives_422(api, numb, field, value) -> None:
    response = api.post("/items", json={**numb, field: value})   # хороший трек, но одно поле испорчено

    assert response.status_code == 422
    assert ["body", field] in _error_fields(response)   # ошибка указывает именно на это поле
    assert api.get("/items").json()["total"] == 0       # и ничего не сохранилось
```

Тест	Что проверяет
POST /items	
`test_create_returns_201_with_new_id_and_location`	201, id = 1, поля без изменений, заголовок Location
`test_create_with_invalid_field_gives_422` ×9	каждое плохое поле → 422 именно про это поле, ничего не сохранилось
`test_create_without_required_field_or_with_id_gives_422`	нет title → 422 «missing»; прислали id → 422 «extra_forbidden»
`test_create_with_taken_spotify_id_gives_409`	второй трек с тем же spotify_id → 409
GET /items/{item_id}	
`test_get_returns_stored_track`	200 и тот же трек, что создали
`test_get_unknown_id_gives_404_not_500`	несуществующий id → 404 и `{"detail": "track 999 not found"}`
`test_get_non_integer_id_gives_422`	`/items/abc` → 422 про `item_id`
GET /items	
`test_list_paginates_with_limit_and_offset`	3 трека: limit=2 → первые два и total 3; offset=2 → третий; offset=10 → пусто
`test_list_with_bad_paging_gives_422` ×4	limit=0, limit=101, offset=-1, limit=ten → 422
GET /items/search	
`test_search_finds_substring_ignoring_case_cyrillic_too`	«nUm» находит Numb, «ГРУППА» находит «Группа крови»
`test_search_without_matches_is_empty_200`	ничего не нашлось → 200 и пустой список, не 404
`test_search_without_valid_query_gives_422` ×3	нет q, пустой q, q длиннее 100 → 422
PUT /items/{item_id}	
`test_replace_changes_fields_and_keeps_created_at`	200, поле изменилось, created_at прежний

`test_replace_unknown_id_gives_404`	несуществующий id → 404
`test_replace_with_invalid_body_gives_422`	плохое тело → 422, трек не изменился
DELETE /items/{item_id}	
`test_delete_gives_204_and_track_disappears`	204 с пустым телом, потом GET → 404
`test_delete_unknown_id_gives_404`	несуществующий id → 404
Общее	
`test_responses_are_ascii_and_keep_cyrillic`	байты ответа — ASCII, после разбора — исходная кириллица; ответ 422 тоже ASCII
`test_openapi_schema_and_swagger_ui_are_served`	`/openapi.json` и `/docs` отвечают, в схеме все адреса и модели
`test_working_database_is_never_opened`	настоящая `get_repository()` ни разу не вызвана

4.10. Тесты хранилища `tests/test_repository.py`

Тест	Что проверяет
`test_data_survives_restart`	создали трек, открыли **новый** объект хранилища на тот же файл — трек на месте (это и есть «переживает перезапуск»)
`test_deleted_id_is_not_given_to_a_new_track`	после удаления трека 1 новый трек получает другой id
`test_search_treats_percent_and_underscore_literally`	поиск «%» находит только «100% Pure Love», поиск «_» — только «snake_case»

4.11. Тесты клиента `tests/test_client.py`

Клиент проверяется двумя способами. **По-настоящему** — против живого uvicorn из фикстуры `live_url`: запросы идут по сети, как в работе. **Подменой сети** — через `httpx.MockTransport`: вместо отправки запроса httpx вызывает нашу функцию, которая отвечает что захотим (503, 404, обрыв связи), и считает попытки.

```
def test_timeout_is_set_and_reported() -> None:
    with socket.socket() as silent:        # "сервер", который принимает соединение и молчит
        silent.bind(("127.0.0.1", 0))
        silent.listen()                    # соединения принимаются, но никто не отвечает
        url = f"http://127.0.0.1:{silent.getsockname()[1]}"
        with LibraryClient(url, timeout=0.3, retries=0) as client:
            with pytest.raises(ServiceUnavailableError, match=r"did not answer within 0\.3 s"):
                client.get(1)              # через 0.3 с должна прийти понятная ошибка, а не вечное ожидание

def test_get_is_retried_on_5xx_until_success(make_client, calls) -> None:
    def flaky(request: httpx.Request) -> httpx.Response:   # "сервер", который сбоит
        if len(calls) < 3:
            return httpx.Response(503, json={"detail": "busy"})   # первые два раза - 503
        return httpx.Response(200, json=_stored())               # третий раз - нормальный трек

    assert make_client(flaky).get(1).title == "Numb"   # клиент дотерпел и вернул трек
    assert calls == ["GET", "GET", "GET"]  # ровно три попытки
```

Тест	Что проверяет
`test_client_covers_every_operation_with_typed_results`	по настоящему HTTP: create, get, list, search, replace, delete; результаты — объекты ItemRead / ItemPage; после удаления — NotFoundError

`test_client_reports_server_side_422_by_field`	плохие данные дошли до сервера → InvalidInputError с полями title и duration_sec
`test_timeout_is_set_and_reported`	молчащий сервер → через 0,3 с ServiceUnavailableError
`test_cli_imports_lab3_json_without_duplicates`	импорт examples/tracks.json: 4 добавлено; повтор: 0 добавлено, 4 пропущено; поиск «группа» находит; весь вывод ASCII
`test_cli_error_is_one_ascii_line_without_traceback`	`get 42` → код 1 и ровно одна строка `error: not found (404): track 42 not found`
`test_cli_checks_input_before_sending`	пустое название → ошибка без обращения к серверу (его по адресу и нет)
`test_get_is_retried_on_5xx_until_success`	503, 503, 200 → трек получен за 3 попытки
`test_post_is_never_retried`	POST при 503 → ServerError после **одной** попытки
`test_4xx_is_not_retried`	404 → NotFoundError после одной попытки
`test_network_failure_after_all_retries_is_readable`	обрыв связи → три попытки → понятный текст «cannot connect ...»
`test_unexpected_body_is_an_api_error`	вместо JSON пришла HTML-страница → ApiError, а не трейсбек

Итого: 33 + 3 + 11 = **47 тестов**. Запуск: `python -m pytest`.

4.12. Настройки `pyproject.toml, requirements.txt, .gitignore`

```
[tool.mypy]
python_version = "3.11"
strict = true                                  # самый строгий режим проверки типов
files = ["app", "client", "tests"]             # что проверять командой python -m mypy
plugins = ["pydantic.mypy"]                    # mypy лучше понимает модели Pydantic

[tool.pytest.ini_options]
testpaths = ["tests"]                          # где искать тесты
pythonpath = ["."]                             # чтобы в тестах работало "from app.main import app"
filterwarnings = ["ignore:Using `httpx` with `starlette.testclient` is deprecated"]
                                               # Starlette 1.x советует форк httpx2; задание требует httpx,
                                               # с ним всё работает - убираем лишний шум из отчёта
```

```
fastapi==0.144.0                               # фреймворк; сам притащит pydantic и starlette
uvicorn==0.54.0                                # сервер
httpx==0.28.1                                  # HTTP-клиент
pytest==9.1.1                                  # тесты
mypy==2.4.0                                    # проверка типов
                                               # точные версии: у всех ставится ровно то, что проверено
```

`.gitignore` не пускает в git окружение `.venv/`, кэши (`__pycache__`, `.pytest_cache`, `.mypy_cache`) и рабочую базу `data/`.

5. Как это связано с заданием

Требование задания	Где сделано
1. Сервис на FastAPI: POST /items (201 + id), GET /items/{id}, GET /items (limit/offset), GET /items/search?q, PUT/DELETE	`app/main.py`: `create_item`, `get_item`, `list_items`, `search_items`, `replace_item`, `delete_item`

2. Отдельные модели ItemCreate и ItemRead, ограничения полей → 422	`app/schemas.py`; `Field(min_length, ge, le, pattern)`, `extra="forbid"`
3. Коды 201 / 200 / 404 (HTTPException) / 422; нет 500 на отсутствующий ресурс	`status_code=201`, `_not_found()`; тесты `..._gives_404_not_500`
4. JSON или SQLite, отдельный слой-репозиторий, данные переживают перезапуск	`app/repository.py`, `Depends(get_repository)`; тест `test_data_survives_restart`
5. Клиент на httpx: timeout, RequestError / HTTPStatusError, raise_for_status, типизированный результат, ASCII-ошибка вместо трейсбека	`client/api.py` — `LibraryClient._request`, `_parse`; `client/cli.py` — `main`
6. Тесты pytest + TestClient на каждый адрес: позитив + негатив (422, 404), изолированное хранилище	`tests/test_api.py`, фикстуры `repo` и `api` в `tests/conftest.py`
7. /docs и /openapi.json; README с venv, pip install, uvicorn, примерами; git; вывод ASCII	FastAPI сам; `README.md`; `AsciiJSONResponse` и `to_ascii`

6. Что сказать на защите

Коротко о программе

«Это REST-сервис музыкальной библиотеки на FastAPI. Ресурс — трек. Есть CRUD, список с пагинацией и поиск по названию. Вход и выход описаны отдельными Pydantic-моделями, поэтому неправильные данные автоматически получают 422. Данные хранятся в SQLite за слоем-репозиторием и переживают перезапуск. Клиент на httpx с таймаутом, проверкой статуса и повторами только для идемпотентных запросов. 47 тестов, у каждого своя временная база. Документация — на /docs.»

Что могут спросить про код

Вопрос	**Ответ**
Почему /items/search выше /items/{item_id}?	Маршруты проверяются по порядку; `{item_id}` поймал бы слово «search» и дал 422.
Откуда берётся 422?	FastAPI проверяет тело, адрес и query по типам и `Field` до вызова функции. Мы сами 422 не пишем.
Почему не 500, когда трека нет?	`repo.get` возвращает `None`, функция бросает `HTTPException(404)`.
Как тесты не трогают рабочие данные?	`app.dependency_overrides` подменяет `get_repository` на базу в `tmp_path`.
Почему поиск через title_fold и instr?	`LIKE` / `lower()` в SQLite не знают регистр кириллицы, а `%` и `_` в LIKE — шаблоны.
Зачем `?` в SQL?	Значения передаются отдельно от запроса — SQL-инъекция невозможна.
Почему POST не повторяется?	Если запрос дошёл, а потерялся ответ, повтор создаст второй трек.
Что будет без timeout?	Зависший сервер заставит клиент ждать вечно.
Что будет без raise_for_status?	404 и 500 считались бы нормальным ответом, ошибка всплыла бы позже и непонятно где.

Контрольные вопросы из лекции

1. Что делает Pydantic-модель в FastAPI, что проверяется, какой код при нарушении? Модель описывает поля, их типы и ограничения. FastAPI разбирает JSON-тело в модель и проверяет: есть ли обязательные поля, тот ли тип, длину строк, диапазон чисел (`ge`/`le`), шаблон, лишние поля (`extra="forbid"`). При нарушении — 422 со списком: какое поле и почему. Лучше ручной проверки: одно описание вместо десятков `if`, единый формат ошибок, документация /docs строится из того же описания, проверку невозможно «забыть» в одном из обработчиков.

2. Чем REST удобнее для публичного API, а gRPC — для внутреннего обмена? REST: JSON читается человеком, легко отлаживать через curl и браузер; HTTP и JSON понимает любой язык и платформа без кодогенерации. gRPC: бинарный Protobuf компактнее и быстрее JSON; строгий контракт в `.proto` с кодогенерацией для обеих сторон; HTTP/2 и нативный стриминг.

3. Зачем очередь, если можно вызвать сервис по REST? Развязка по времени — тяжёлая работа не держит запрос (сразу 202 и id задачи); сглаживание пиков — очередь работает буфером; устойчивость — упал воркер, задача вернётся в очередь и её возьмёт другой; ретраи с задержкой при временных сбоях.

4. Как FastAPI-эндпоинт соотносится с HTTP поверх сокетов на первом курсе? Раньше: `accept`, `recv` в цикле, поиск `\r\n\r\n`, разбор строки запроса и заголовков, `Content-Length`, декодирование, проверка полей вручную, сборка байтов ответа. Теперь сокет и HTTP разбирает uvicorn, а FastAPI выбирает функцию по методу и пути, достаёт параметры и тело, проверяет их, превращает ответ в JSON, ставит код и строит документацию.

5. Что такое идемпотентность, какие методы идемпотентны, как это влияет на ретраи и очереди? Повтор операции даёт тот же результат, что и одно выполнение. GET, PUT, DELETE — идемпотентны, POST — нет. Поэтому наш клиент повторяет GET/PUT/DELETE, но не POST. В очереди с доставкой «хотя бы раз» сообщение может прийти дважды, и обработчик должен проверять, не сделано ли уже, — как наш импорт по уникальному `spotify_id` не создаёт дублей.

6. Зачем клиенту timeout и raise_for_status? Без timeout зависший сервер держит клиента вечно и занимает соединения. Без `raise_for_status` ответы 404/500 идут дальше как успешные: код падает позже на `resp.json()` или молча работает с ошибкой вместо данных.

7. Как по коду (4xx или 5xx) решить, повторять ли запрос? 4xx — виноват клиент, повтор бесполезен, надо исправить запрос: 422 на пустое название, 404 на несуществующий трек. 5xx — проблема сервера, она может пройти: 503 при перегрузке — повторить с растущей паузой (у нас 0,5 с, потом 1 с).